

C++ FILE I/O — THEORY ONLY

Exam-format notes extracted from the provided past questions • Programs intentionally excluded

A. Extracted Non-Program Theory Topics

Topic	Covered from the question sets
Stream / file stream	Meaning of stream; file stream; role of streams in C++
File-stream classes	ifstream, ofstream, fstream; stream class hierarchy
Opening a file	Methods, syntax, examples, open modes
File modes / ios flags	in, out, app, ate, trunc, binary, and combined flags
EOF handling	End-of-file condition and proper checking
Stream errors	good(), eof(), fail(), bad(), operator/state checking
File-operation errors	Common errors encountered during file operations
File access	Types of file access: sequential and random/direct
File access pointers	get pointer, put pointer, seekg, seekp, tellg, tellp
I/O formatting	Formatting output; width, precision, fill, alignment, flags
Manipulators	endl, setw, setprecision, fixed, scientific, setfill, left/right
Stream class hierarchy	ios_base → basic_ios → istream/ostream → iostream

Important: Repeated theory questions were merged. No programming solutions are included, even when the original question also contained a program-writing part.

1. What is a Stream?

Definition: A stream is a flow of data between a program and an input/output device. For input, data flows toward the program; for output, data flows away from the program.

Basic idea:

Input: **Device/File** → **Stream** → **Program**

Output: **Program** $\rightarrow$ **Stream** $\rightarrow$ **Device/File**

Examples: `cin` is the standard input stream, `cout` is the standard output stream, and file streams such as `ifstream` and `ofstream` connect the program to files.

2. What is a File Stream?

A file stream is a stream used to transfer data between a C++ program and a file. It allows the program to read data from a file and/or write data to a file.

Header file: `<fstream>`

Memory trick: Stream = data flow; File stream = data flow involving a file.

3. Classes Used for File Stream

Class	Purpose	Typical use
ifstream	Input file stream	Read from a file
ofstream	Output file stream	Write to a file
fstream	File stream for both input and output	Read and write a file

Trick: I = Input = ifstream → read; **O = Output = ofstream** → write; **F = File = fstream** → both.

4. Stream Class Hierarchy

[illegible]

Explanation: `ios_base` provides basic stream-related facilities and formatting/state support. `basic_ios` manages the common stream state. `istream` supports input and `ostream` supports output. `iostream` supports both. File-specific classes such as `ifstream`, `ofstream`, and `fstream` provide file input/output.

5. How Can a File Be Opened in C++?

A file can be opened either by passing the filename to the stream object's constructor or by using the **open()** member function.

Syntax using constructor:

```
ifstream fin("file.dat");
ofstream fout("file.dat");
fstream file("file.dat", ios::in | ios::out);
```

Syntax using open():

```
stream_object.open("filename", mode);
```

Example: `ifstream fin; fin.open("student.dat", ios::in);`

Normal sequence: Open → Check → Read/Write → Close.

6. File Opening Modes / ios Flags

Mode	Meaning
<code>ios::in</code>	Open for input / reading.
<code>ios::out</code>	Open for output / writing.
<code>ios::app</code>	Append; all output is written at the end of the file.
<code>ios::ate</code>	Open and initially position the file pointer at the end; it can be moved afterward.
<code>ios::trunc</code>	Discard existing file contents when opening for output.
<code>ios::binary</code>	Open in binary mode.

Combining modes: Use the bitwise OR operator `|`, e.g. `ios::in | ios::out`.

Trick: `in` = read, `out` = write, `app` = add at end, `ate` = start at end, `trunc` = clear old contents, `binary` = binary representation.

7. End-of-File (EOF) Handling

EOF means **End Of File**. It indicates that there is no more data available to read from the file.

A good way to handle EOF is to test the result of the input operation itself. For example, while reading records, the loop can continue only while the read operation succeeds.

Exam point: Do not normally use `while (!fin.eof())` as the main record-reading condition because EOF is generally detected only after an attempted read reaches the end. Prefer testing the input operation.

Related function: `eof()` returns true when the stream's EOF state is set.

8. Testing Stream Errors

Function	Meaning
<code>good()</code>	Returns true when no error state is set.
<code>eof()</code>	Returns true when end-of-file has been reached / EOF state is set.
<code>fail()</code>	Returns true when an input/output operation fails or a format-related failure occurs.
<code>bad()</code>	Returns true when a serious input/output error occurs.

Quick memory: `good` = okay, `eof` = end, `fail` = operation failed, `bad` = serious error.

Common check: `if (!fin) ...` or `if (fin.fail()) ...` can be used to detect an unsuccessful opening/operation.

9. Errors Encountered During File Operations

Error/problem	Explanation
File not found	The specified file does not exist when a read operation expects it.
Unable to open file	The program lacks access or the file/path cannot be opened.
Invalid operation	An operation is attempted in an unsuitable stream state or mode.
Read failure	The requested data cannot be extracted successfully.
Write failure	Data cannot be written successfully.
EOF condition	There is no more input available from the file.
Permission/access problem	The program is not allowed to access the file.
Bad I/O state	A serious underlying input/output error may have occurred.

10. Types of File Access

1. Sequential access: Data is processed in order, from the beginning toward the end. To reach a later record, earlier data is normally passed over first. It is simple and common for ordinary record processing.

2. Random/direct access: The file position can be moved directly to a required location, allowing a particular part of a file to be accessed without processing every preceding record.

Memory trick: Sequential = **one after another**; Random = **jump to a location**.

11. File Access Pointers

A file access pointer is the current position in a file from which data is read or to which data is written.

Pointer	Meaning
---------	---------

Get pointer	Position used for input/reading.
Put pointer	Position used for output/writing.
seekg()	Moves the get/read pointer.
seekp()	Moves the put/write pointer.
tellg()	Returns the current get/read position.
tellp()	Returns the current put/write position.

Trick: g = get = read; p = put = write. So **seekg/tellg** work with the get pointer and **seekp/tellp** work with the put pointer.

12. I/O Stream Formatting

Formatting means controlling how input or output data appears on the console/file stream, such as field width, number of decimal places, alignment, fill character, and number format.

Feature	Purpose / example
setw(n)	Sets the width of the next output field.
setprecision(n)	Controls precision; with fixed, it commonly specifies digits after the decimal point.
setfill(ch)	Sets the character used to fill extra field width.
fixed	Displays floating-point values in fixed-point notation.
scientific	Displays floating-point values in scientific notation.
left	Left-aligns output within the field.
right	Right-aligns output within the field.

Typical header: `#include <iomanip>` is used for manipulators such as `setw`, `setprecision`, and `setfill`.

13. Different Techniques for Formatting I/O Streams

C++ supports formatting through **stream manipulators**, member functions, and stream flags.

Manipulators: `endl`, `setw`, `setprecision`, `setfill`, `fixed`, `scientific`, `left`, `right`.

Stream member functions / flags: functions and flags can control width, precision, fill character, alignment, base representation, and floating-point notation.

Exam conclusion: Formatting makes output organized, readable, aligned, and suitable for tables/reports.

14. Stream Class Hierarchy + ios Flags: Combined Exam Answer

For a question asking to 'explain stream class hierarchy by highlighting ios flags', write the hierarchy first, then define the important flags used to control stream behavior.

Flag	Purpose
<code>ios::in</code>	Input/read mode
<code>ios::out</code>	Output/write mode
<code>ios::app</code>	Append output at the end
<code>ios::ate</code>	Initially position at end
<code>ios::trunc</code>	Truncate existing contents
<code>ios::binary</code>	Binary mode

15. Short-Note: File Access Pointers and Their Manipulators

File access pointers: `get` pointer and `put` pointer indicate the current input and output positions.

Manipulators/functions: `seekg()` and `tellg()` operate on the `get` pointer; `seekp()` and `tellp()` operate on the `put` pointer.

Memory: `g` = `get/read`, `p` = `put/write`; `seek` = `move`, `tell` = `report position`.

16. Quick Revision Sheet

Question keyword	Write these points
Stream	Flow of data between program and input/output device.
File stream	Stream used for file input/output; .
Classes	ifstream = read, ofstream = write, fstream = both.
Open file	Constructor or open(); syntax: object.open(filename, mode).
File modes	in, out, app, ate, trunc, binary.
EOF	End Of File; test input operation / eof state.
Stream errors	good, eof, fail, bad.
File access	Sequential and random/direct.
Pointers	get/put; seekg/seekp; tellg/tellp.
Formatting	setw, setprecision, setfill, fixed, scientific, left, right.
Hierarchy	ios_base → basic_ios → istream/ostream → iostream; file stream classes build on these.

Exam strategy: For 2–3 marks, give definition + 2–3 key points. For 5–6 marks, add a table/diagram, syntax, and a small example where requested. For a 10-mark theory question, combine the relevant sections above in the same order as the question.